

Practical Week-6

Task 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
lenovo@Manya:~$ mkdir -p ~/FIFO_Project
lenovo@Manya:~$ cd ~/FIFO_Project
lenovo@Manya:~/FIFO_Project$ pwd
/home/lenovo/FIFO_Project
lenovo@Manya:~/FIFO_Project$ nano server.c
lenovo@Manya:~/FIFO_Project$ nano client.c
lenovo@Manya:~/FIFO_Project$ gcc server.c -o server
lenovo@Manya:~/FIFO_Project$ gcc client.c -o client
lenovo@Manya:~/FIFO_Project$ ls
client  client.c  server  server.c
lenovo@Manya:~/FIFO_Project$ cd ~/FIFO_Project
lenovo@Manya:~/FIFO_Project$ ./server
Server started...
Waiting for client message...
Client: Hello Server
Response sent to client.
Client: How are you?
Response sent to client.
```

```
lenovo@Manya
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define CLIENT_FIFO "/tmp/client_fifo"

int main()
{
    int server_fd, client_fd;
    char buffer[100];

    // Create named pipes
    mkfifo(SERVER_FIFO, 0666);
    mkfifo(CLIENT_FIFO, 0666);

    printf("Server started...\n");
    printf("Waiting for client message...\n");

    while (1)
    {
        // Open server FIFO for reading
        server_fd = open(SERVER_FIFO, O_RDONLY);

        if (server_fd == -1)
        {
            perror("Error opening server FIFO");
            exit(1);
        }

        // Read message from client
        memset(buffer, 0, sizeof(buffer));
        read(server_fd, buffer, sizeof(buffer));
    }
}
```

```

// Read message from client
memset(buffer, 0, sizeof(buffer));
read(server_fd, buffer, sizeof(buffer));

printf("Client: %s\n", buffer);

close(server_fd);

// Process the message
char response[150];

snprintf(response, sizeof(response),
          "Server received: %s", buffer);

// Open client FIFO for writing
client_fd = open(CLIENT_FIFO, O_WRONLY);

if (client_fd == -1)
{
    perror("Error opening client FIFO");
    exit(1);
}

write(client_fd, response, strlen(response) + 1);

close(client_fd);

printf("Response sent to client.\n");
}

return 0;
}

```

```

lenovo@Manya:~$ cd ~/FIFO_Project
lenovo@Manya:~/FIFO_Project$ ./client
Enter message: Hello Server
Server: Server received: Hello Server
lenovo@Manya:~/FIFO_Project$ ./client
Enter message: How are you?
Server: Server received: How are you?
lenovo@Manya:~/FIFO_Project$

```

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define CLIENT_FIFO "/tmp/client_fifo"

int main()
{
    int server_fd, client_fd;
    char message[100];
    char response[150];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    // Remove newline
    message[strcspn(message, "\n")] = '\0';

    // Open server FIFO
    server_fd = open(SERVER_FIFO, O_WRONLY);

    if (server_fd == -1)
    {
        perror("Error opening server FIFO");
        exit(1);
    }

    // Send message to server
    write(server_fd, message, strlen(message) + 1);

    close(server_fd);

    // Open client FIFO for reading response
    client_fd = open(CLIENT_FIFO, O_RDONLY);
```

```

    if (client_fd == -1)
    {
        perror("Error opening client FIFO");
        exit(1);
    }

    // Read response
    memset(response, 0, sizeof(response));
    read(client_fd, response, sizeof(response));

    printf("Server: %s\n", response);

    close(client_fd);

    return 0;
}

```

Task 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```

lenovo@Manya:~$ mkdir -p ~/Signal_Project
lenovo@Manya:~$ cd ~/Signal_Project
lenovo@Manya:~/Signal_Project$ nano signal.c
lenovo@Manya:~/Signal_Project$ gcc signal.c -o signal
lenovo@Manya:~/Signal_Project$ ls
signal  signal.c
lenovo@Manya:~/Signal_Project$ ./signal
Signal handling program started.
Process ID (PID): 1367
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received! (Ctrl+C)
Program is running...
^C
SIGINT received! (Ctrl+C)
Program is running...
Program is running...

```

```
lenovo@Manya
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received! (Ctrl+C)\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received! Terminating program...\n");
        exit(0);
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received! User-defined signal handled.\n");
    }
}

int main()
{
    // Register signal handlers
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Signal handling program started.\n");
    printf("Process ID (PID): %d\n", getpid());

    printf("Waiting for signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```